

Extension Justification

How I Extended my Code

I redesigned my game to be more extensible by changing from a hardcoded, linear phase system (Spawn → Move → Build → Repeat) to an extensible **GodCard** abstraction. Each **GodCard** defines its own sequence of **TurnActions**, such as selecting a worker, moving, building, or applying special powers. These **TurnActions** are queued by a **TurnIterator**, which allows for more **TurnActions** to be added dynamically. During a player's turn, the **Game** class delegates turn control to the current **GodCard**'s action sequence, and each player can have a different GodCard modifying their turn behavior. This allows for easy addition of new god powers without modifying the core game logic, only requiring the creation of new **GodCard** subclasses. I used a **DefaultLogicCard** of type **GodCard** for the default rules of the game, and I actually used it to delegated certain tasks for ease of reuse.

Justification and Design Principles

This redesign follows the **Open/Closed Principle** (open for extension, closed for modification) by making the game extensible without needing changes to the core **Game** logic for every new god. I also applied the **Strategy Pattern**, with each **GodCard** acting as a different "strategy" for how a player's turn is organized. This reduced coupling between player-specific behavior and global game state. A tradeoff with my approach is the upfront complexity of setting up actions, iterators, and card classes, but the benefit is that it becomes much easier to add and test new gods independently. An alternative method I previously used was to have set phases for Move and Build. However, I ran into problems when some of the bonus God powers wanted deviations from this pattern, such as Prometheus allowed players to build before moving. I also sort of used the **Template Pattern** when I was making some of my logic cards by providing reusable methods for default behavior because I did not want to keep repeating certain methods over and over whenever I was not changing the default behavior. Lastly, I used **delegation** in many of my **God Cards** in their **TurnActions**. I used a **DefaultLogicCard** delegator to perform the standard behavior whenever I only needed to make a small change to that behavior. An alternative to this would have been inheritance, but I felt that delegation was better because I could have more **encapsulation** by not relying on the specifics of the super class implementations.